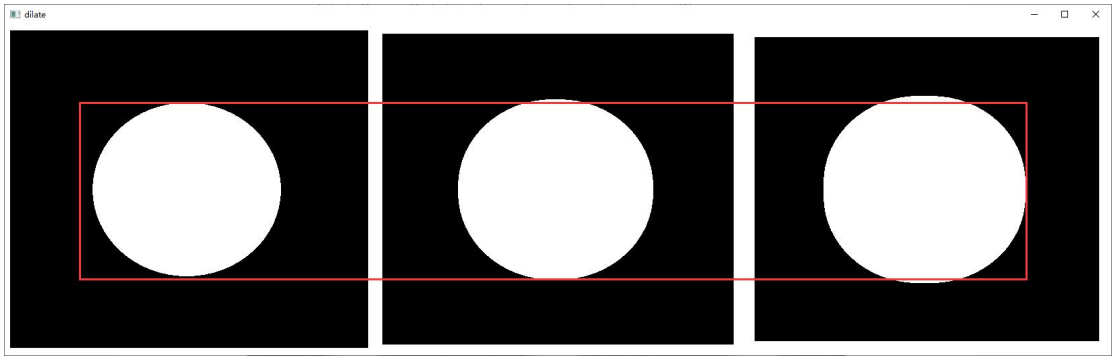
项 8.项目 2 答题卡识别-任务 1.图像形态学操作

1.任务 1.1.1：图像膨胀操作
2.任务 1.1.2：使用膨胀操作消除椒盐噪声
3.任务 1.1.3 使用膨胀操作获取图像轮廓
4.任务 1.2.1 使用腐蚀操作消除图像毛刺
5.任务 1.2.2 使用腐蚀操作消除图像高光亮点
6.任务 1.2.3 使用腐蚀操作消除图像连接
7.任务 1.2.4 使用腐蚀操作消除椒盐噪声
8.任务 1.2.5 使用腐蚀操作得到物体轮廓
9.任务 1.3.1 使用开运算消除物体毛刺
10.任务 1.3.2 使用开运算分离图像中的圆盘和直线任务
11.任务 1.3.3 使用开运算分离图像中的垂直线
12.任务 1.4.1 使用闭运算筛选图像_保留大孔去掉小孔
13.任务 1.4.2 使用闭运算消除椒盐噪声
14.任务 1.4.3 使用闭运算删除图像中的直线
15.任务 1.5.1 礼帽运算
16.任务 1.5.2 黑帽运算

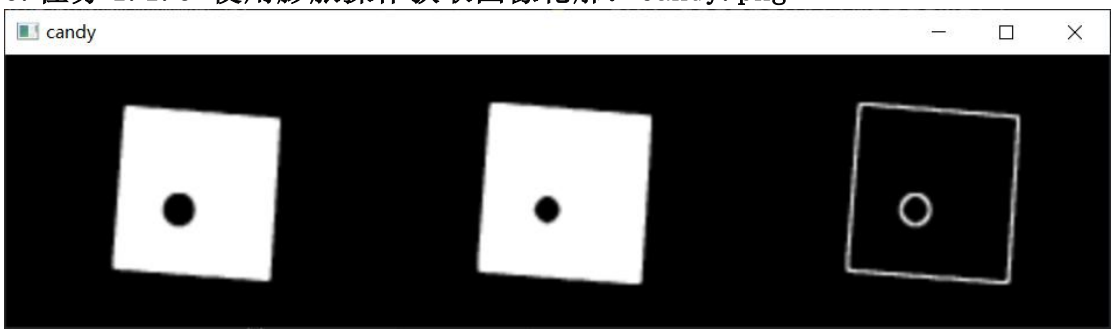
1.任务 1.1.1：图像膨胀操作

```
img1 = cv2.imread('../image/dige.png', 0)
img2 = cv2.imread('../image/pie.png', 0)
#膨胀操作
def dilate(img):
    kernel = np.ones((3, 3), np.uint8)
    img_dilate1 = cv2.dilate(img, kernel, iterations=1)
    img_dilate2 = cv2.dilate(img, kernel, iterations=2)
    res =np.hstack((img, img_dilate1, img_dilate2))
    cv2.imshow('dilate', res)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
dilate(img1)
```



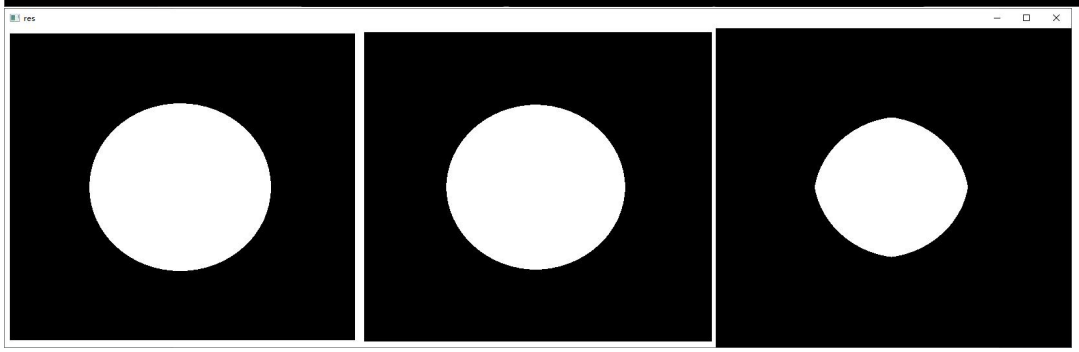
2. 任务 1.1.2：使用膨胀操作消除椒盐噪声：facenoise.png

3. 任务 1.1.3 使用膨胀操作获取图像轮廓：candy.png



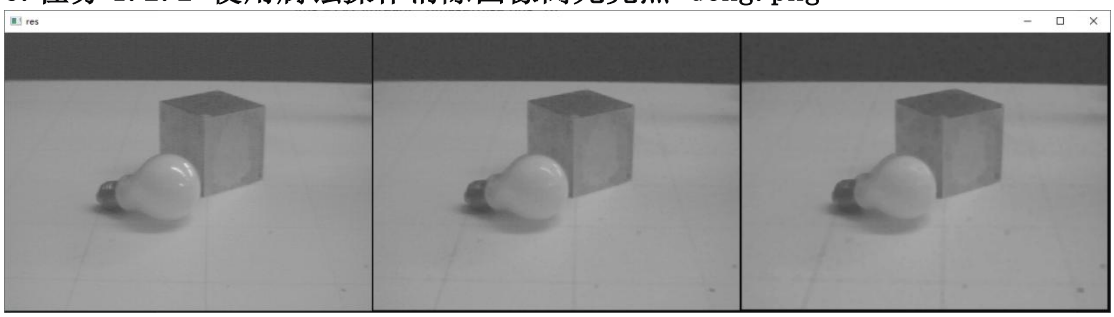

```
import cv2
import numpy as np
img = cv2.imread('../image/candy.png', 0)
def dilate():
    kernel = np.ones((3, 3), np.uint8)
    img_dilate2 = cv2.dilate(img, kernel, iterations=2)
    img_candy=img_dilate2-img
    candy =np.hstack((img, img_dilate2, img_candy))
    cv2.imshow('candy', candy)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
dilate()
```

4. 任务 1.2.1 使用腐蚀操作消除图像毛刺：dige.png，pie.png

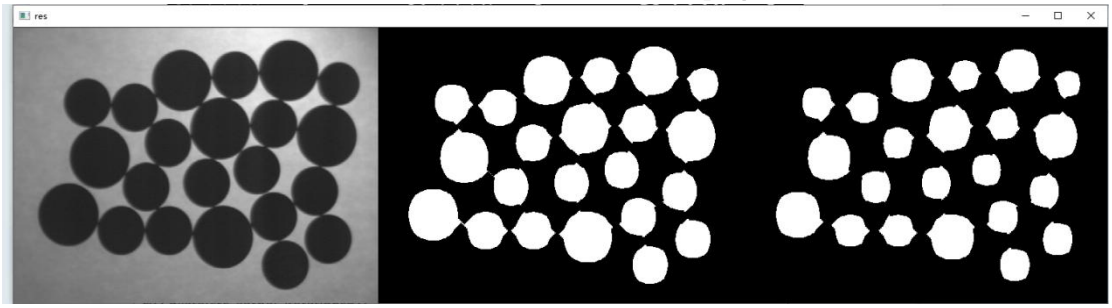



```
import cv2
import numpy as np
img = cv2.imread('../image/dige.png', 0)
img2 = cv2.imread('../image/pie.png', 0)
def erode(img):
    kernel = np.ones((5, 5), np.uint8)
    erosion1 = cv2.erode(img, kernel, iterations=1) #iterations 表示迭代次数
    erosion2 = cv2.erode(img, kernel, iterations=2)
    res =np.hstack((img, erosion1, erosion2))
    cv2.imshow('res', res)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
erode(img)
```

5. 任务 1.2.2 使用腐蚀操作消除图像高光亮点 deng.png



6. 任务 1.2.3 使用腐蚀操作消除图像连接 coin.png

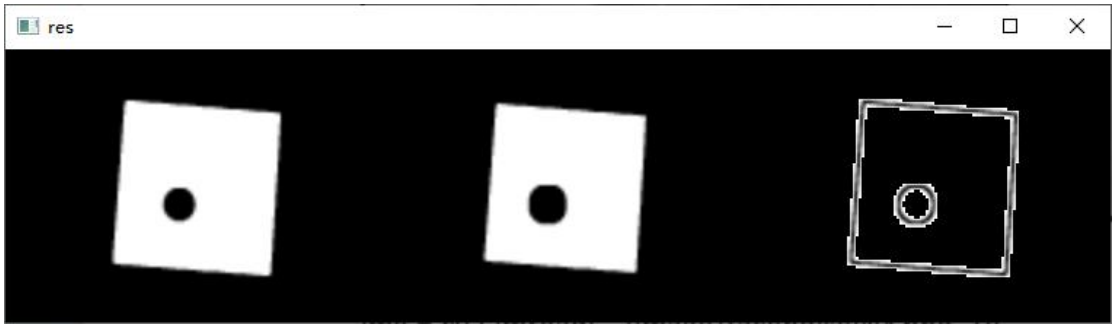


```
import cv2
import numpy as np
img = cv2.imread('../image/coin.png', 0)
def erode(op):
    if(op == 1) :
        kernel = np.ones((3,3), np.uint8)
    else :
        kernel = cv2.getStructuringElement(cv2.MORPH_CROSS, (5, 5))
    ret,thresh1 = cv2.threshold(img,90,255,cv2.THRESH_BINARY_INV)
    erosion1 = cv2.erode(thresh1, kernel, iterations=4) #iterations 表示迭代次数
    erosion2 = cv2.erode(thresh1, kernel, iterations=6)
    res =np.hstack((img, erosion1, erosion2))
    cv2.imshow('res', res)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
erode(0)
```

7. 任务 1.2.4 使用腐蚀操作消除椒盐噪声

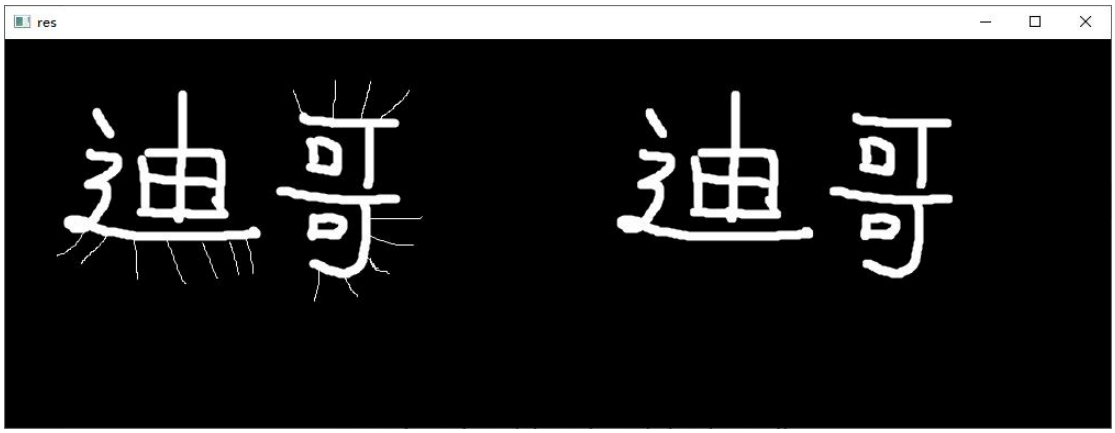
```
import cv2
import numpy as np
img = cv2.imread('../image/facenoiseINY.png', 0)
def erode():
    kernel = np.ones((3,3),np.uint8)
    erosion1 = cv2.erode(img, kernel, iterations=1) #iterations 表示迭代次数
    erosion2 = cv2.erode(img, kernel, iterations=2)
    res =np.hstack((img, erosion1))
    cv2.imshow('res', res)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
erode()
```

8. 任务 1.2.5 使用腐蚀操作得到物体轮廓



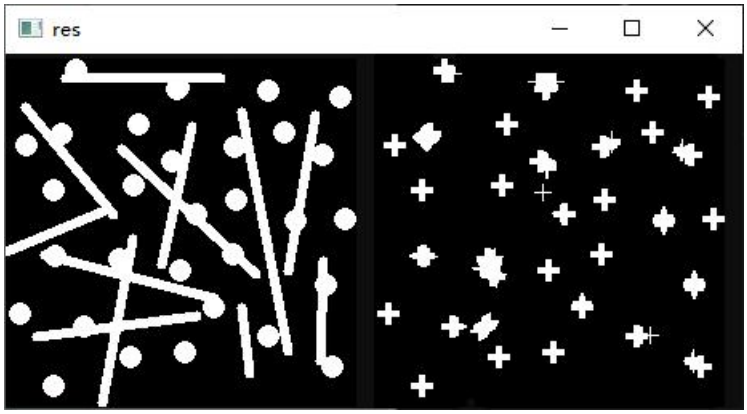

```
import cv2
import numpy as np
img = cv2.imread('../image/candy.png', 0)
def erode():
    kernel = np.ones((3,3),np.uint8)
    erosion1 = cv2.erode(img, kernel, iterations=1) #iterations 表示迭代次数
    erosion2 = cv2.erode(img, kernel, iterations=2)
    img_candy = erosion2 - img
    res =np.hstack((img, erosion2,img_candy))
    cv2.imshow('res', res)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
erode()
```

9. 任务 1.3.1 使用开运算消除物体毛刺



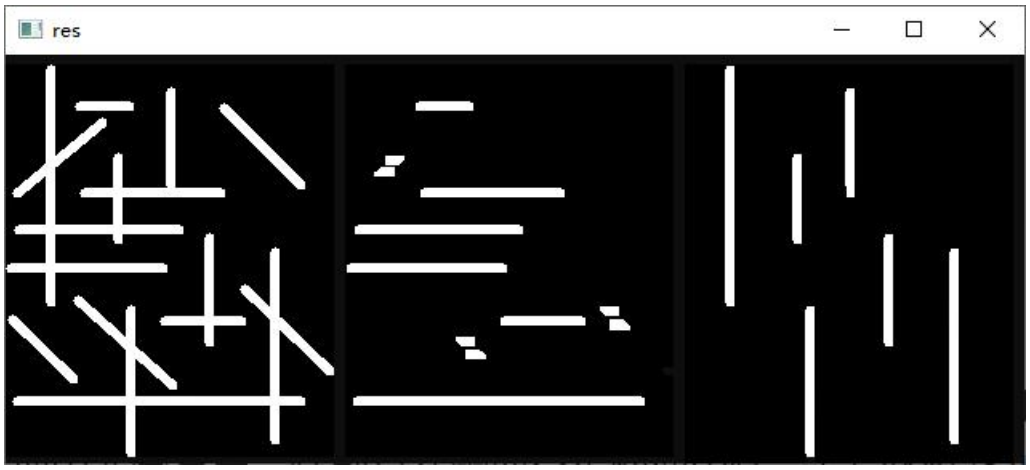

```
import cv2
import numpy as np
img = cv2.imread('../image/dige.png', 0)
def open_art():
    kernel = np.ones((5,5),np.uint8)
    opening = cv2.morphologyEx(img,cv2.MORPH_OPEN,kernel)
    res =np.hstack((img, opening))
    cv2.imshow('res', res)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
open_art()
```

10. 任务 1.3.2 使用开运算分离图像中的圆盘和直线任务：cirrec.jpg




```
import cv2
import numpy as np
img = cv2.imread('../image/cirrec.jpg', 0)
def open_cir():
    kernel_cir = cv2.getStructuringElement(cv2.MORPH_CROSS, (11,11))
    opening = cv2.morphologyEx(img,cv2.MORPH_OPEN,kernel_cir)
    res =np.hstack((img, opening))
    cv2.imshow('res', res)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
open_cir()
```

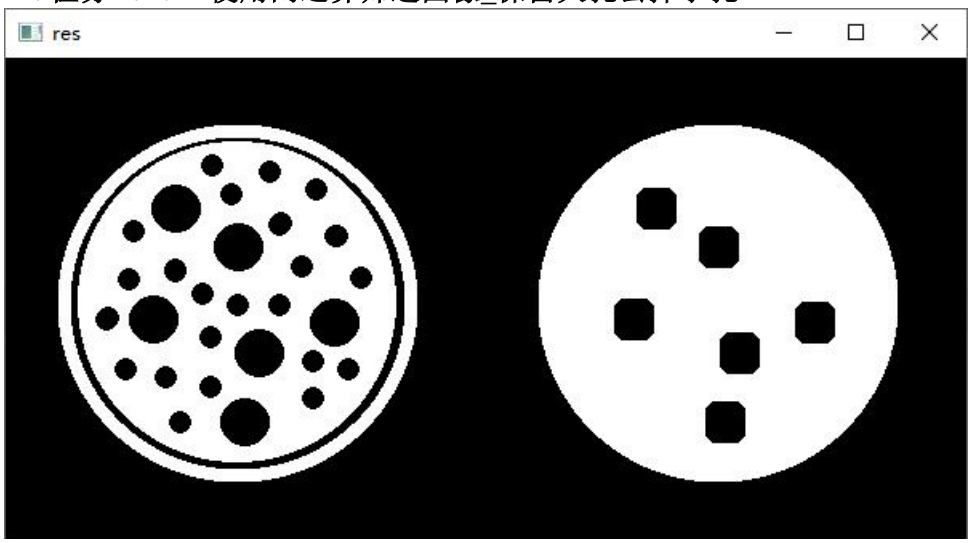
11. 任务 1.3.3 使用开运算分离图像中的垂直线




```
import cv2
import numpy as np
img = cv2.imread('../image/hs.jpg', 0)
def get_line():
    #提取水平线：3*9：细长的矩形，其长边为 9 个像素，短边为 3 个像素
    #更容易与水平方向的线条对齐
    kernel1 = np.ones((3,9),np.uint8)
    #提取垂直线：9*3：宽短的矩形，其长边为 9 个像素，短边为 3 个像素
    #更容易与垂直方向的线条对齐
    kernel2 = np.ones((9,3),np.uint8)
    open1 = cv2.morphologyEx(img,cv2.MORPH_OPEN,kernel1)
    open2 = cv2.morphologyEx(img,cv2.MORPH_OPEN,kernel2)
    res =np.hstack((img, open1,open2))
    cv2.imshow('res', res)
    cv2.waitKey(0)
```

```
    cv2.destroyAllWindows()
get_line()
```

12. 任务 1.4.1 使用闭运算筛选图像_保留大孔去掉小孔



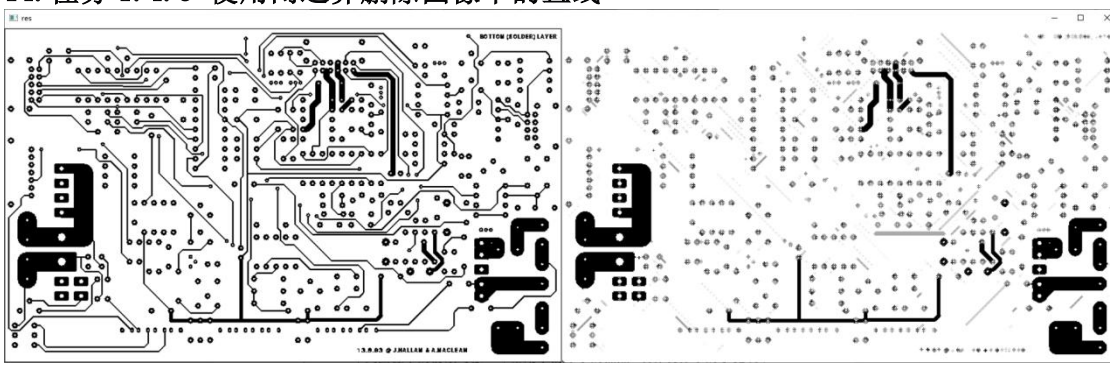

```
import cv2
import numpy as np
img = cv2.imread('../image/pank.png', 0)
def close(img):
    kernel1 = np.ones((17,17),np.uint8)
    kernel2= cv2.getStructuringElement(cv2.MORPH_CROSS,(17,17))
    closing = cv2.morphologyEx(img,cv2.MORPH_CLOSE,kernel2)
    res =np.hstack((img, closing))
    cv2.imshow('res', res)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
close(img)
```

13. 任务 1.4.2 使用闭运算消除椒盐噪声

```
import cv2
import numpy as np
img = cv2.imread('../image/facenoise.png', 0)
```

```
def close():
    kernel1 = np.ones((3,3),np.uint8)
    open1 = cv2.morphologyEx(img,cv2.MORPH_CLOSE,kernel1,iterations=1)
    res =np.hstack((img, open1))
    cv2.imshow('res', res)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
close()
```

14. 任务 1.4.3 使用闭运算删除图像中的直线




```
import cv2
import numpy as np
img = cv2.imread('../image/dl.png', 0)
def close():
    kernel1 = cv2.getStructuringElement(cv2.MORPH_CROSS,(5,5))
    open1 = cv2.morphologyEx(img,cv2.MORPH_OPEN,kernel1,iterations=1)
    open2 = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel1, iterations=1)
    res =np.hstack((img, open2))
    cv2.imshow('res', res)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
close()
```

15.任务 1.5.1 礼帽运算

```
import cv2
import numpy as np
img = cv2.imread('../image/dige.png', 0)
def tophat():
    kernel = np.ones((7, 7), np.uint8)
    open = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
    tophat = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, kernel)
    res1 =np.hstack((img,open, tophat))
    cv2.imshow('res', res1)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
tophat()
```

16.任务 1.5.2 黑帽运算

```
img = cv2.imread('../image/dige.png', 0)
img2 = cv2.imread('../image/j.png', 0)
def blackhat(img):
    kernel = np.ones((5, 5), np.uint8)
    close = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)
    blackhat =    cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, kernel)
    res2 = np.hstack((img, close,blackhat))
    cv2.imshow('res', res2)
    cv2.waitKey(0)
blackhat(img2)
```

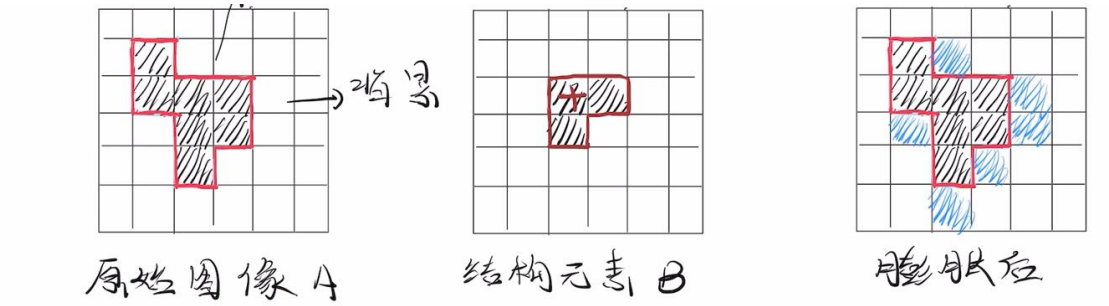
一、形态学操作原理

形态学操作主要应用于消除噪声、边界提取、区域填充、连通分量提取、凸壳、细化、粗化等；同时还能够分割出独立的图像元素，或者图像中相邻的元素，并求取图像中明显的极大值区域和极小值区域，最终求取图像梯度。

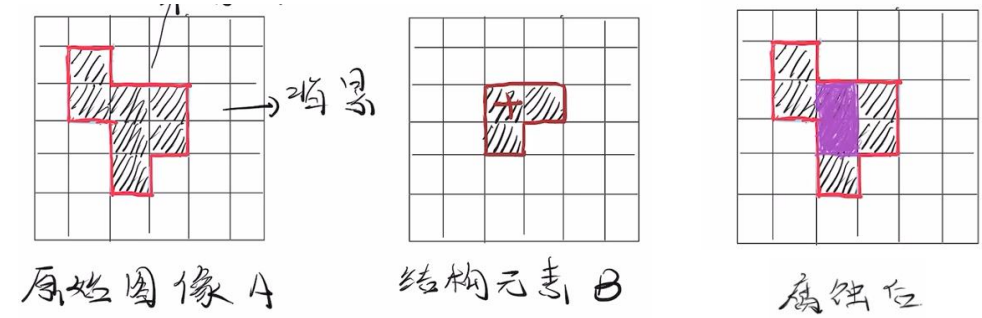
应用范围：二值图像、灰度图像中应用，彩色图像上效果不明显，所以在图像形态学操作之前，可将彩色图像转换成二值图像。

操作	作用	说明
膨胀	对白色而言，变胖，扩大图像中的前景对象	使图像中的白色区域（高亮度区域）扩大，常用于消除小的黑点、填补物体中的小孔或连接断裂的部分。
腐蚀	对白色而言，变瘦，缩小图像中的前景对象	使图像中的白色区域缩小，常用于消除小的白色噪点或分割连接在一起的物体。
开启	先腐蚀再膨胀	消除小的物体或噪点，平滑物体的边界。开启操作可以去掉较小的对象，同时保持较大对象的形态。
闭合	先膨胀再腐蚀	填补物体中的小孔，平滑物体的边界。闭合操作有助于连接小的断裂部分，消除物体中的小孔。

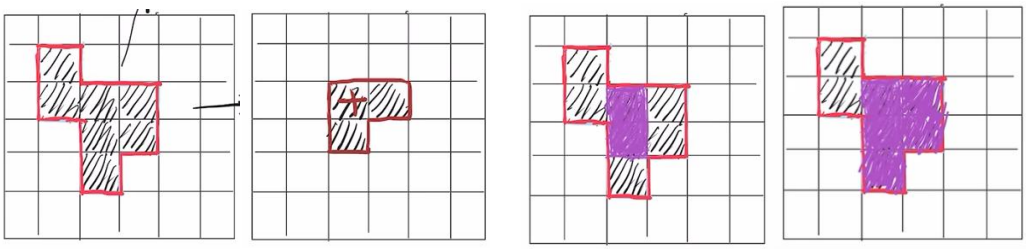
1、膨胀：取或



2、腐蚀，取与

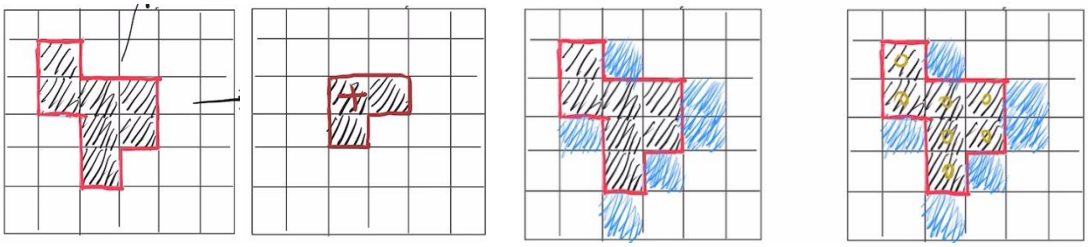


3、开运算：先腐蚀、后膨胀



原图　　结构元素　　先腐蚀　　后膨胀

4、闭运算：先膨胀、后腐蚀



原图　　结构元素　　先膨胀　　后腐蚀

5、举例

A

0	0	0	0	0	0	0
0	0	0	1	1	0	0
0	1	1	1	1	1	1
0	0	1	1	1	0	1
0	1	1	1	1	1	1
0	0	0	0	0	0	0

原图

B

0	1	0
1	1	1
0	1	0

结构元素

0	0	0	1	1	0	0	0
0	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1
0	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
0	1	1	1	1	1	1	0

膨胀后

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0
0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

腐蚀后

二、具体使用方法

1. 图像膨胀

cv2.dilate 是 OpenCV 中用于图像膨胀（Dilation）操作的函数。膨胀是一种基于形态学的图像处理技术，常用于二值图像（黑白图像）或灰度图像，主要用于扩大图像中的白色区域（前景区域），同时缩小黑色区域（背景区域）。它在图像去噪、边缘增强、连接断裂部分等方面非常有用。

cv2.dilate(src, kernel, iterations=1)
src：输入图像，通常是灰度图像或二值图像。 kernel：膨胀操作的结构元素（卷积核）。默认 3*3 矩阵，可以修改。 iterations：膨胀操作的迭代次数，默认值为 1。增加迭代次数可以进一步扩大白色区域。 返回值：返回膨胀后的图像，与输入图像具有相同的尺寸和类型。

```
kernel = np.ones((3,3), np.uint8)
```

np.ones：创建一个指定形状和数据类型的全 1 数组
(3,3)：数组的形状参数，表示创建一个 3×3 的二维数组。
np.uint8：表示数组中的每个元素都是无符号 8 位整数（范围是 0 到 255）

```
[[1, 1, 1],
 [1, 1, 1],
 [1, 1, 1]]
```

```
kernel = cv2.getStructuringElement(cv2.MORPH_CROSS, (5, 5))
```

cv2.getStructuringElement:生成结构元素,是一个二维数组，通常用于定义图像形态学操作的邻域范围和形状。
cv2.MORPH_CROSS：形状参数，表示生成的结构元素是一个十字形（cross-shaped）。
(5, 5)：结构元素的大小，表示生成的十字形结构元素的高度和宽度均为 5。

```
[[0, 0, 1, 0, 0],
 [0, 0, 1, 0, 0],
 [1, 1, 1, 1, 1],
 [0, 0, 1, 0, 0],
 [0, 0, 1, 0, 0]]
```

中心点 (2, 2) 是结构元素的锚点（anchor point），在形态学操作中，该点会与图像中的像素对齐。
值为 1 的位置表示结构元素的邻域范围，值为 0 的位置则不参与操作。
支持多种形状如矩形、椭圆形、十字形等。

2. 图像腐蚀

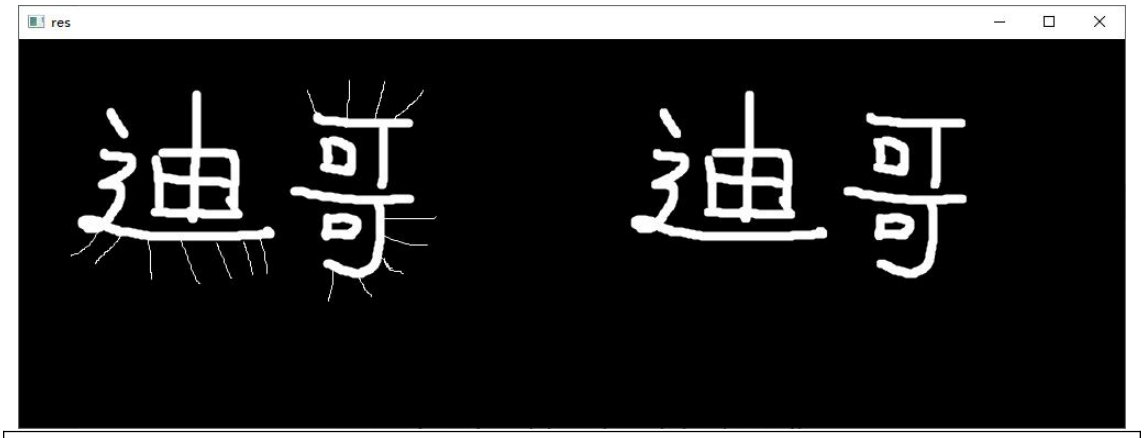
cv2.erode 图像腐蚀操作的函数。腐蚀是一种数学形态学操作，主要用于减少或缩小图像中的白色（高亮度）区域，同时扩大黑色（低亮度）区域。它常用于消除小的噪点、断开物体之间的连接、去除细小的线条或突出物等。

```
cv2.erode(src, kernel, iterations=1)
```

src：输入图像，通常是灰度图像或二值图像。
kernel：膨胀操作的结构元素（卷积核）。默认 3*3 矩阵，可以修改。
iterations：膨胀操作的迭代次数，默认值为 1。增加迭代次数可以进一步扩大白色区域。

返回值：返回腐蚀后的图像，与输入图像具有相同的尺寸和类型。

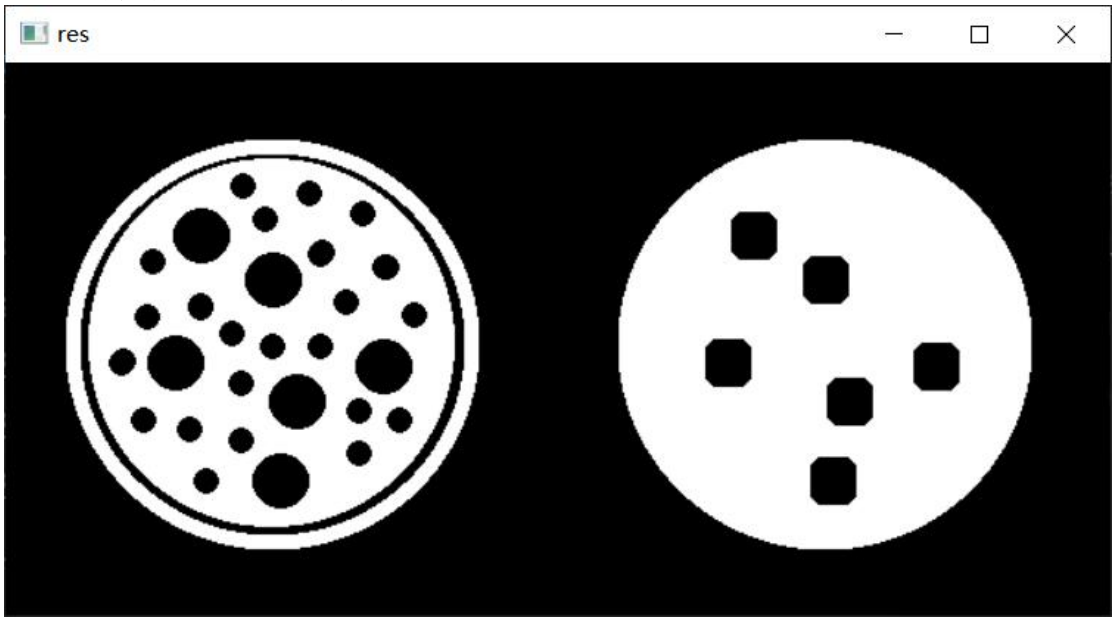
3. 开运算:先腐蚀、后膨胀



cv2.morphologyEx(src, op, kernel, dst=None, anchor=None, iterations=1, borderType=None, borderValue=None)

src：输入图像，通常是灰度图像或二值图像。
op：
形态学操作类型，可选值包括：
cv2.MORPH_ERODE：腐蚀操作。
cv2.MORPH_DILATE：膨胀操作。
cv2.MORPH_OPEN：开运算（先腐蚀后膨胀）：去除小噪声点。
cv2.MORPH_CLOSE：闭运算（先膨胀后腐蚀）：填补小孔。
cv2.MORPH_GRADIENT：形态学梯度（膨胀和腐蚀的差值）：突出物体的边缘。
cv2.MORPH_TOPHAT：顶帽运算（原图像减去开运算结果）：突出比周围亮的区域。
cv2.MORPH_BLACKHAT：黑帽运算（闭运算结果减去原图像）：突出比周围暗的区域。
kernel：膨胀操作的结构元素（卷积核）。默认 3*3 矩阵，可以修改。
iterations：膨胀操作的迭代次数，默认值为 1。增加迭代次数可以进一步扩大白色区域。
返回值：返回处理后的图像，与输入图像具有相同的尺寸和类型。

4. 闭运算:先膨胀、后腐蚀



5. 礼帽运算

图像开运算或闭运算之后，亮度会发生变化，可以用礼帽运算和黑帽运算解决。

开运算：先腐蚀，再膨胀，　　结果是去除刺的内容，不带刺

礼帽运算=原图像-开运算结果　　结果是刺

dst = tophat(src,element)=src-open(src,element)

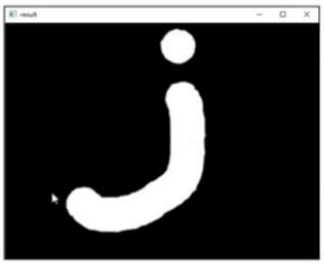
原始图像　　开运算图像　　礼帽图像

6. 黑帽运算

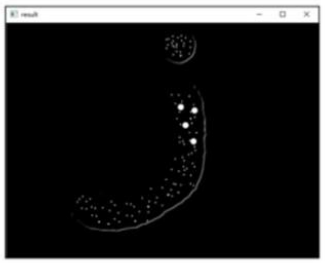
闭运算：先膨胀，再腐蚀，　　结果是去除空洞的内容，带刺

黑帽=闭运算结果-原图像

原始图像



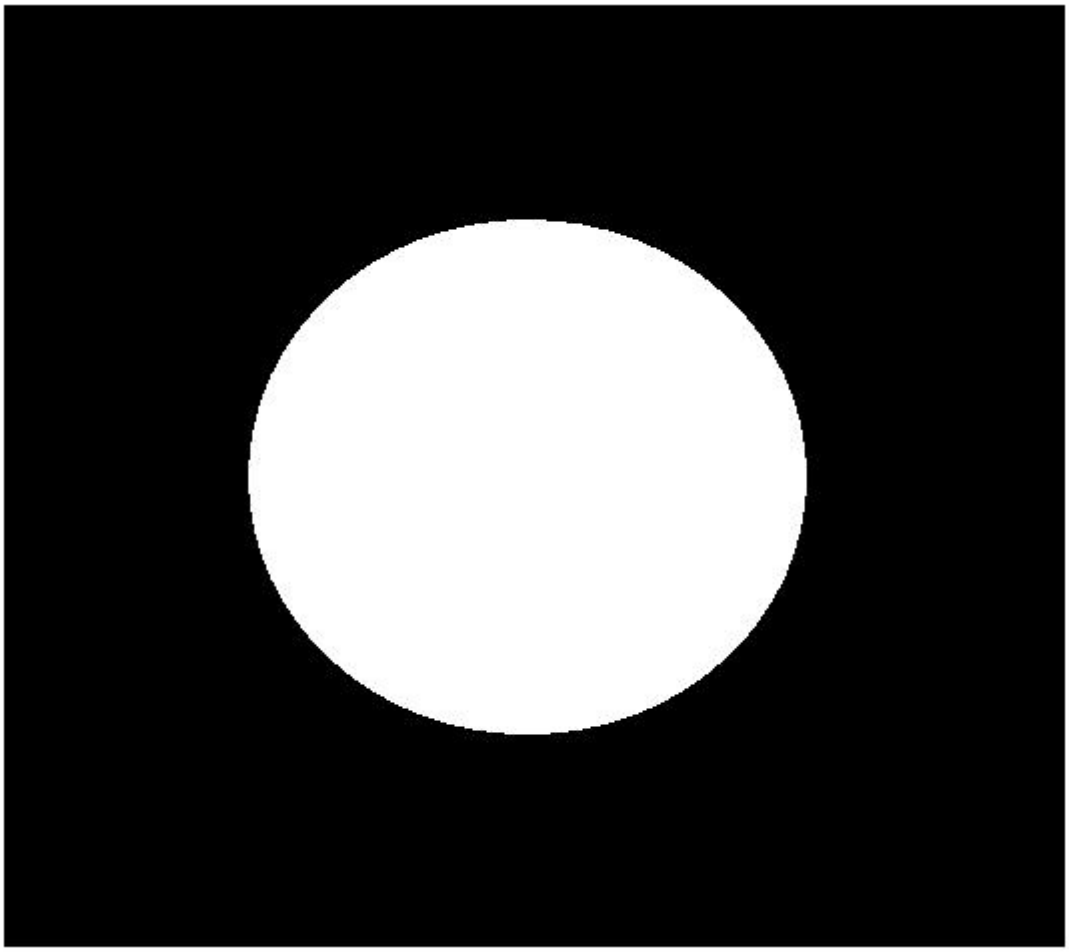
闭运算图像



黑帽图像

作业：
一、膨胀与腐蚀的对比实验：
1.分别对下面两张图像执行以下操作：
①膨胀 1 次、3 次、5 次。
②腐蚀 1 次、3 次、5 次。
2.使用两种结构元素：
①np.ones((3,3), np.uint8)（矩形）
②cv2.getStructuringElement(cv2.MORPH_CROSS, (3,3))（十字形）
3.将原始图像与所有结果图像拼接显示，并保存为一张大图。并回答问题
①随着迭代次数增加，膨胀和腐蚀的效果如何变化？什么时候会出现图像“过度膨胀”或“过度腐蚀”？
②对于 dige.png 中的毛刺，腐蚀和膨胀哪个更容易消除？为什么？
③十字形结构元素与矩形结构元素的结果有什么差异？请结合图像具体区域说明。

迪哥



二、开运算与闭运算模拟图像中的去噪与修补

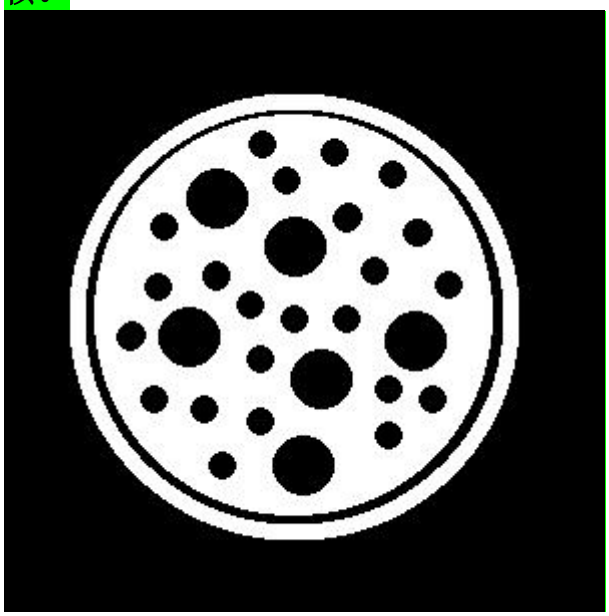
1.噪声去除：对人脸图片自行添加椒盐噪声，并分别使用：

①开运算（注意：要把彩色图片转换成灰度图）

②闭运算（注意：要把彩色图片转换成灰度图）

③中值滤波（cv2.medianBlur，核大小 5）比较三种方法，找出最适合去除椒盐噪声的一种，并说明理由。

2.孔洞填补：对下图使用闭运算，尝试以下不同大小的核：(5,5)、(11,11)、(17,17)观察哪些孔被填满，哪些大孔仍然保留。解释原因。注：使用正方形核。



3.假设你有一张答题卡局部图像（自己用画图软件简单画 3×3 个实心圆，部分圆内添加小黑点，部分圆之间有轻微粘连），请设计一个开运算+闭运算的组合流程，同时实现：

①去除圆点外部的独立小噪点

②填补圆点内部的小空洞

③不破坏正常圆点的形状